

Set 23: Multiple Tables with Pandas

Skill 23.1: Describe how data can be stored across multiple tables to reduce redundancy

Skill 23.2: Identify information by connecting data across multiple tables

Skill 23.3: Apply the *merge()* method to combine tables

Skill 23.4: Rename columns to support merging tables.

Skill 23.5: Compare inner and outer merges

Skill 23.6: Concatenate DataFrames

Skill 23.1: Describe how data can be stored across multiple tables to reduce redundancy

Skill 23.1 Concepts

To efficiently store data, we often spread related information across multiple tables.

For instance, imagine that we own an e-commerce business and we want to track the products that have been ordered from our website.

We could have one table with all the following information,

- `order_id`
- `customer_id`
- `customer_name`
- `customer_address`
- `customer_phone_number`
- `product_id`
- `product_description`
- `product_price`
- `quantity`
- `timestamp`

However, a lot of this information would be repeated. If the same customer makes multiple orders, that customer's name, address, and phone number will be reported multiple times. If the same product is ordered by multiple customers, then the product price and description will be repeated. This will make our orders table big and unmanageable.

So instead, we can split our data into three tables,

- orders would contain the information necessary to describe an order: *order_id*, *customer_id*, *product_id*, *quantity*, and *timestamp*
- products would contain the information to describe each product: *product_id*, *product_description* and *product_price*
- customers would contain the information for each customer: *customer_id*, *customer_name*, *customer_address*, and *customer_phone_number*

In this lesson, we will learn the Pandas commands that help us work with data stored in multiple tables.

Skill 23.1 Exercise 1

Skill 23.2: Identify information by connecting data across multiple tables

Skill 23.2 Concepts

Suppose we have the following three tables that describe our eCommerce business:

Orders				
order_id	customer_id	product_id	quantity	timestamp
1	2	3	1	2017-01-01
2	2	2	3	2017-01-01
3	3	1	1	2017-01-01
4	3	2	2	2017-02-01
5	3	3	3	2017-02-01
6	1	4	2	2017-03-01
7	1	1	1	2017-02-02
8	1	4	1	2017-02-02

Products		
product_id	description	price
1	thing-a-ma-jig	5
2	whatcha-ma-call-it	10
3	doo-hickey	7
4	gizmo	3

Customers			
customer_id	customer_name	address	phone_number
1	John Smith	123 Main St.	212-123-4567
2	Jane Doe	456 Park Ave.	949-867-5309
3	Joe Schmo	798 Broadway	112-358-1321

If we just look at the orders table, we can't really tell what's happened in each order. However, if we refer to the other tables, we can get a more complete picture.

Let's examine the order with an *order_id* of 1. It was purchased by Customer 2. To find out the customer's name, we look at the customers table and look for the item with a *customer_id* value of 2. We can see that Customer 2's name is Jane Doe and that she lives at 456 Park Ave.

Doing this kind of matching is called *merging* two DataFrames.

Skill 23.2 Exercise 1

Skill 23.3: Apply the *merge()* method to combine tables

Skill 23.3 Concepts

It is easy to do this kind of matching for one row, but what if we wanted to do this for many rows, or an entire column?

The pandas *merge()* method looks for columns that are common between two DataFrames and then looks for rows where those column's values are the same. It then combines the matching rows into a single row in a new table.

Below is an example,

Data			
Sales			Targets
	month	revenue	month target
0	January	300	0 January 310
1	February	290	1 February 270
2	March	310	2 March 300
3	April	325	3 April 350
4	May	475	4 May 475
5	June	495	5 June 500
Code			
<pre>sales_targets = pd.merge(sales, targets) display(sales_targets)</pre>			
Output			
	month	revenue	target
0	January	300	310
1	February	290	270
2	March	310	300
3	April	325	350
4	May	475	475
5	June	495	500

Merge can also be applied to join two or more tables by chaining commands,

Data									
sales			targets		men_women				
month	revenue		month	target	month	men women			
0	January	300	0	January	310	0	January	30	35
1	February	290	1	February	270	1	February	29	35
2	March	310	2	March	300	2	March	31	29
3	April	325	3	April	350	3	April	32	28
4	May	475	4	May	475	4	May	47	50
5	June	495	5	June	500	5	June	49	45
Code									
combined = sales.merge(targets).merge(men_women)									
display(combined)									
Output									
month	revenue	target	men	women					
0	January	300	310	30	35				
1	February	290	270	29	35				
2	March	310	300	31	29				
3	April	325	350	32	28				
4	May	475	475	47	50				
5	June	495	500	49	45				

Combining the data allows us to answer different questions than we couldn't answer if it were separated. Below is an example,

Data					
	month	revenue	target	men	women
0	January	300	310	30	35
1	February	290	270	29	35
2	March	310	300	31	29
3	April	325	350	32	28
4	May	475	475	47	50
5	June	495	500	49	45
Question					
Is revenue greater for the months when more women's t-shirts are sold?					
Code					
<pre>results = combined[(combined.revenue > combined.target) & (combined.women > combined.men)] display(results)</pre>					
Output					
	month	revenue	target	men	women
1	February	290	270	29	35

[Skill 23.3 Exercise 1](#)

Skill 23.4: Rename columns to support merging tables.

Skill 23.4 Concepts

In the previous example, the `merge()` function “knew” how to combine tables based on the columns that were the same between two tables. For instance, `products` and `orders` both had a column called `product_id`. This won't always be true when we want to perform a merge.

Generally, the `products`, `customers`, and `orders` DataFrames would not have the columns `product_id`, `customer_id`, or `order_id`. Instead, they would all be called `id`.

Customers			
id	customer_name	address	phone_number
1	John Smith	123 Main St.	212-123-4567
2	Jane Doe	456 Park Ave.	949-867-5309
3	Joe Schmo	798 Broadway	112-358-1321

Products		
id	description	price
1	thing-a-ma-jig	5
2	whatcha-ma-call-it	10
3	doo-hickey	7
4	gizmo	3

Orders				
id	customer_id	product_id	quantity	timestamp
1	2	3	1	2017-01-01
2	2	2	3	2017-01-01
3	3	1	1	2017-01-01
4	3	2	2	2017-02-01
5	3	3	3	2017-02-01
6	1	4	2	2017-03-01
7	1	1	1	2017-02-02
8	1	4	1	2017-02-02

How would this affect our merges???

Because the *id* columns would mean something different in each table, our default merges would be wrong.

One way that we could address this problem is to use *rename()* to rename the columns for our merges. In the example below, we will rename the column *id* to *customer_id*, so that orders and customers have a common column for the merge.

Data									
customers					orders				
	id	customer_name	address	phone_number		id	product_id	customer_id	quantity timestamp
0	1	John Smith	123 Main St.	212-123-4567	0	1	3	2	1 2017-01-01
1	2	Jane Doe	456 Park Ave.	949-867-5309	1	2	2	2	3 2017-01-01
2	3	Joe Schmo	798 Broadway	112-358-1321	2	3	1	3	1 2017-01-01
					3	4	2	3	2 2016-02-01
					4	5	3	3	3 2017-02-01
					5	6	4	1	2 2017-03-01
					6	7	1	1	1 2017-02-02
					7	8	4	1	1 2017-02-02

Code

```
customers.rename(columns = {"id":"customer_id"}, inplace = True)
customer_orders = customers.merge(orders)
display(customer_orders)
```

Output

	customer_id	customer_name	address	phone_number	id	product_id	quantity	timestamp
0	1	John Smith	123 Main St.	212-123-4567	6	4	2	2017-03-01
1	1	John Smith	123 Main St.	212-123-4567	7	1	1	2017-02-02
2	1	John Smith	123 Main St.	212-123-4567	8	4	1	2017-02-02
3	2	Jane Doe	456 Park Ave.	949-867-5309	1	3	1	2017-01-01
4	2	Jane Doe	456 Park Ave.	949-867-5309	2	2	3	2017-01-01
5	3	Joe Schmo	798 Broadway	112-358-1321	3	1	1	2017-01-01
6	3	Joe Schmo	798 Broadway	112-358-1321	4	2	2	2016-02-01
7	3	Joe Schmo	798 Broadway	112-358-1321	5	3	3	2017-02-01

Skill 23.4 Exercise 1

Skill 23.5: Compare inner and outer merges

Skill 23.5 Concepts

In our previous examples, there were always matching values when we were performing our merges. What happens when that isn't true?

Imagine that our products table is out of date and is missing the newest product: Product 5. What happens when someone orders it?

Data																																																																	
orders						products																																																											
<table><thead><tr><th></th><th>id</th><th>product_id</th><th>customer_id</th><th>quantity</th><th>timestamp</th></tr></thead><tbody><tr><td>0</td><td>1</td><td>3</td><td>2</td><td>1</td><td>2017-01-01</td></tr><tr><td>1</td><td>2</td><td>2</td><td>2</td><td>3</td><td>2017-01-01</td></tr><tr><td>2</td><td>3</td><td>5</td><td>1</td><td>1</td><td>2017-01-01</td></tr><tr><td>3</td><td>4</td><td>2</td><td>3</td><td>2</td><td>2016-02-01</td></tr><tr><td>4</td><td>5</td><td>3</td><td>3</td><td>3</td><td>2017-02-01</td></tr></tbody></table>							id	product_id	customer_id	quantity	timestamp	0	1	3	2	1	2017-01-01	1	2	2	2	3	2017-01-01	2	3	5	1	1	2017-01-01	3	4	2	3	2	2016-02-01	4	5	3	3	3	2017-02-01	<table><thead><tr><th></th><th>product_id</th><th>description</th><th>price</th></tr></thead><tbody><tr><td>0</td><td>1</td><td>thing-a-ma-jig</td><td>5</td></tr><tr><td>1</td><td>2</td><td>whatcha-ma-call-it</td><td>10</td></tr><tr><td>2</td><td>3</td><td>doo-hickey</td><td>7</td></tr><tr><td>3</td><td>4</td><td>gizmo</td><td>3</td></tr></tbody></table>					product_id	description	price	0	1	thing-a-ma-jig	5	1	2	whatcha-ma-call-it	10	2	3	doo-hickey	7	3	4	gizmo	3
	id	product_id	customer_id	quantity	timestamp																																																												
0	1	3	2	1	2017-01-01																																																												
1	2	2	2	3	2017-01-01																																																												
2	3	5	1	1	2017-01-01																																																												
3	4	2	3	2	2016-02-01																																																												
4	5	3	3	3	2017-02-01																																																												
	product_id	description	price																																																														
0	1	thing-a-ma-jig	5																																																														
1	2	whatcha-ma-call-it	10																																																														
2	3	doo-hickey	7																																																														
3	4	gizmo	3																																																														

Code																																																															
``` merged_df = orders.merge(products) display(merged_df) ```																																																															
Output																																																															
		id	product_id	customer_id	quantity	timestamp	description	price		---	----	------------	-------------	----------	------------	--------------------	-------		0	1	3	2	1	2017-01-01	doo-hickey	7		1	5	3	3	3	2017-02-01	doo-hickey	7		2	2	2	2	3	2017-01-01	whatcha-ma-call-it	10		3	4	2	3	2	2016-02-01	whatcha-ma-call-it	10										

Check out the results above.. The order with id = 3 is missing!

When we merge two DataFrames whose rows don't match perfectly, we lose the unmatched rows.

This type of merge (where we only include matching rows) is called an *inner merge*. There are other types of merges that we can use when we want to keep information from the unmatched rows.

Suppose that two companies, Company A and Company B have just merged. They each have a list of customers, but they keep slightly different data. Company A has each customer's name and email. Company B has each customer's name and phone number. They have some customers in common, but some are different.

If we wanted to combine the data from both companies without losing the customers who are missing from one of the tables, we could use an *Outer Join*. An *Outer Join* would include all rows from both tables, even if they don't match. Any missing values are filled in with None or nan (which stands for "Not a Number").



Data		
company_a		company_b
name	email	name      phone
Sally Sparrow	<a href="mailto:sally.sparrow@gmail.com">sally.sparrow@gmail.com</a>	Peter Grant      212-345-6789
Peter Grant	<a href="mailto:pgrant@yahoo.com">pgrant@yahoo.com</a>	Leslie May      626-987-6543
Leslie May	<a href="mailto:leslie_may@gmail.com">leslie_may@gmail.com</a>	Aaron Burr      303-456-7891
Code		
<code>pd.merge(company_a, company_b, how='outer')</code>		
Output		
name	email	phone
Sally Sparrow	<a href="mailto:sally.sparrow@gmail.com">sally.sparrow@gmail.com</a>	nan
Peter Grant	<a href="mailto:pgrant@yahoo.com">pgrant@yahoo.com</a>	212-345-6789
Leslie May	<a href="mailto:leslie_may@gmail.com">leslie_may@gmail.com</a>	626-987-6543
Aaron Burr	nan	303-456-7891

### Skill 23.5 Exercise 1

## Skill 23.6: Concatenate DataFrames

### Skill 23.6 Concepts

Sometimes, a dataset is broken into multiple tables. For instance, data is often split into multiple CSV files so that each download is smaller.

When we need to reconstruct a single DataFrame from multiple smaller DataFrames, we can use the *concat()* method. This method only works if all of the columns are the same in all of the DataFrames.

Below is an example,

Data			
df_1		df_2	
name	email	name	email
Katja Obinger	k.obinger@gmail.com	Jean Gray	jgray@netscape.net
Alison Hendrix	alisonH@yahoo.com	Scott Summers	ssummers@gmail.com
Cosima Niehaus	cosi.niehaus@gmail.com	Kitty Pryde	kitkat@gmail.com
Rachel Duncan	rachelduncan@hotmail.com	Charles Xavier	cxavier@hotmail.com
Code			
<pre>pd.concat([df1, df2])</pre>			
Output			
name	email		
Katja Obinger	k.obinger@gmail.com		
Alison Hendrix	alisonH@yahoo.com		
Cosima Niehaus	cosi.niehaus@gmail.com		
Rachel Duncan	rachelduncan@hotmail.com		
Jean Gray	jgray@netscape.net		
Scott Summers	ssummers@gmail.com		
Kitty Pryde	kitkat@gmail.com		
Charles Xavier	cxavier@hotmail.com		

*merge* combines DataFrames by **matching rows using shared columns**, adding information across tables.

*concat* combines DataFrames by **stacking rows or columns together** without matching values.

In short, use **merge** to **match data** and **concat** to **append data**.

### Skill 23.6 Exercise 1